

Modula-2 / Oberon-2 — Reference

A subset of **Wirth's successors to Pascal** compiled with `lex` + `yacc` + `cc` (`oberon.l` / `oberon.y` lower the source to C; `cc` lowers the C to .NET IL). One front end serves both languages: there is no dialect switch, and no separate `.mod` / `.ob` handling — the grammar accepts a single `MODULE Name; ... END Name.` compilation unit and whichever features you use decide which language you are writing. Modula-2's `FROM InOut IMPORT WriteString` and Oberon-2's `Out.String` both work in the same file, as do Oberon-2's `ELSIF`, `LOOP` / `EXIT`, `RETURN`, record extension and type-bound procedures.

These languages are **case-sensitive** and their keywords are **UPPERCASE**; the operator set is Wirth's later one (`#` for "not equal", `&` for `AND`, `~` for `NOT`, `{ }` for set constructors). The module body becomes `main`; every `PROCEDURE` becomes a `public static` method `o_<name>` on `CProgram`, and every `RECORD` becomes a C `struct` with a vtable pointer at offset 0 so that record extension is layout-compatible.

```
oberon prog.mod          # compile to prog.exe (native .NET executable)
oberon prog.mod -o app.exe # choose the output name
```

Quick Reference

```
MODULE Demo;                                (* nested (* comments *) are fine *)
IMPORT Out;                                (* or: FROM InOut IMPORT WriteString, WriteLn; *)
CONST
  Max = 10;                                (* CONST takes any constant expression *)
  Name = "Oberon";                          (* a string constant *)
TYPE
  Point = RECORD x, y: INTEGER END;
  Vec = ARRAY 5 OF INTEGER;  (* 0-based *)
  Node = POINTER TO NodeDesc;
  NodeDesc = RECORD (Point) next: Node END;  (* RECORD (Base) = extension *)
VAR
  n: INTEGER;  x: REAL;  c: CHAR;  ok: BOOLEAN;  s: SET;

PROCEDURE Square(k: INTEGER): INTEGER;      (* result via RETURN *)
BEGIN RETURN k * k END Square;
PROCEDURE Swap(VAR a, b: INTEGER);          (* VAR = by reference *)
VAR t: INTEGER;
BEGIN t := a; a := b; b := t END Swap;
PROCEDURE (p: Node) Show;                  (* type-bound procedure *)
BEGIN Out.Int(p.x, 0) END Show;

BEGIN
  n := 42;  c := CHR(65);  s := {1, 3, 5..9};
  Out.String("n = "); Out.Int(n, 4); Out.Ln;
  IF n > 0 THEN ... ELSIF n = 0 THEN ... ELSE ... END;
  CASE n OF 1: ... | 2, 3: ... | 4..6: ... ELSE ... END;
  FOR n := 1 TO 10 BY 2 DO ... END;
  WHILE n > 0 DO ... END;      REPEAT ... UNTIL n = 0;
  LOOP ... IF done THEN EXIT END END;
  IF 3 IN s THEN ... END
END Demo.
```

Data types

INTEGER / **LONGINT** / **SHORTINT** / **CARDINAL** (all 32-bit), **REAL** / **LONGREAL** (both 64-bit double), **CHAR**, **BOOLEAN** (**TRUE** / **FALSE**), and **SET** (a 0..255 bitset held by a runtime handle). **Arrays** are **ARRAY n OF T**, **0-based**, and nest for more dimensions (**ARRAY 3 OF ARRAY 3 OF INTEGER**), indexed **a[i]** or **a[i, j]**. **Records** (**RECORD ... END**) nest and assign by value; **RECORD (Base)** *extends* a record, inheriting its fields. **Pointers** are **POINTER TO T**, allocated with **NEW**; **p^** dereferences and **p.f** auto-dereferences.

Literals: **123**, **1.5**, **1.5E3**, and **"text"** or **'text'** (both give a string). There is no character literal — build a **CHAR** with **CHR(n)** or by indexing an **ARRAY OF CHAR**.

Statements / Commands

Assignment (`:=`), procedure calls, `IF ... THEN ... ELIF ... ELSE ... END`, `CASE ... OF ... | ... ELSE ... END` (labels may be single integers, comma lists, or `lo..hi` ranges), `WHILE ... DO ... END`, `REPEAT ... UNTIL`, `FOR i := a TO b [BY step] DO ... END`, `LOOP ... END` with `EXIT`, and `RETURN [expr]`.

Functions

A `PROCEDURE` with a result type (`PROCEDURE Square(k: INTEGER): INTEGER`) is a function and yields its value with `RETURN expr`; without a result type it is a plain procedure. Parameters are by value by default; `VAR` parameters are by reference, so `Swap(a, b)` exchanges the caller's variables. Recursion works. Every procedure becomes a static method, so a procedure must be declared before it is called textually — except for recursion into itself.

Type-bound procedures (Oberon-2's methods) are written with a receiver: `PROCEDURE (f: Figure) Area(): INTEGER;`. The receiver may be a record or a pointer to one, and inside the body its fields and other type-bound procedures are reachable both through the receiver name and unqualified. Every type-bound procedure is **virtual**: it gets a vtable slot, an override reuses the base's slot, `NEW` installs the vtable pointer, and calls dispatch dynamically — including from a base-typed variable and from a non-overridden base procedure.

Built-in functions. `ORD`, `CHR`, `ABS`, `ODD`, `CAP`, `LEN`, `SHORT`, `LONG`, `ENTIER`, `TRUNC`, `FLOAT`. **Built-in procedures.** `INC`, `DEC`, `NEW`, `HALT`, `INCL`, `EXCL`.

Input / Output

Output only, through built-in pseudo-modules — no import is actually required, but writing `IMPORT Out;` or `FROM InOut IMPORT ...;` is accepted and idiomatic. Both spellings map to the same emitter:

Oberon-2	Modula-2	Effect
<code>Out.String(s)</code>	<code>WriteString(s)</code>	write a string
<code>Out.Int(n, w)</code>	<code>WriteInt(n, w)</code> , <code>WriteCard(n, w)</code>	write an integer in field width <code>w</code>
<code>Out.Real(x, w)</code>	<code>WriteReal(x, w)</code>	write a real in field width <code>w</code>
<code>Out.Char(c)</code>	<code>WriteChar(c)</code> , <code>Write(c)</code>	write one character
<code>Out.Ln</code>	<code>WriteLn</code>	end the line

`Out.Int(n, 0)` means "no padding". A `BOOLEAN` passed to `Out.Int` prints `TRUE` / `FALSE`. Reals print in `%g` style. Modula-2's unqualified forms may also be written as `Terminal.WriteString(...)` etc.

There is **no input** and **no file I/O**.

Graphics

None. Activity 8 draws a chart with text.

Notes

- **Case-sensitive**, keywords **UPPERCASE**: `begin` is an identifier, `BEGIN` is a keyword.
- `#` and `<` both mean "not equal"; `&` is **AND**, `~` is **NOT**.
- `DIV` is integer division and `MOD` the remainder (truncated: `-7 MOD 3` is `-1`); `/` always yields a **REAL**.
- Comments are `(* ... *)` and nest properly. String literals have **no escape sequences** — a `\` inside `"..."` will not parse.
- Every block closes with `END`, and a `MODULE` / `PROCEDURE` repeats its own name after `END`.
- `IMPORT` and `FROM ... IMPORT ...` clauses are parsed and discarded; an export mark (`*` after a declared name, as in `PROCEDURE Sum*`) is accepted and ignored.
- Set membership is `IN`; union/difference/intersection are `+`, `-`, `*`.

Subset boundaries

A working Wirth-family core — modules, structured types, the full statement set, and Oberon-2's record extension with virtual type-bound procedures — with these honest gaps and defects:

- **No string variables.** Only string *literals* and string *constants* (`CONST Name = "Oberon"`) are text; `+`, `=`, `<`, `LEN` and `Out.String` work on those. An `ARRAY n OF CHAR` is a genuine character array: `s := "hello"` compiles but produces garbage, and `Out.String(s)` prints a number. Fill and read such an array one `CHAR` at a time.
 - **No nested procedures.** A `PROCEDURE` inside another `PROCEDURE` is accepted by the grammar but the emitted C does not compile.
 - **ARRAY n OF T needs a literal length.** A named constant length (`ARRAY N OF INTEGER`) is silently approximated as **64** elements. `ARRAY OF T` (an open array parameter) is parsed but has no length information.
 - **The library modules are names only.** `Out`, `InOut`, `Terminal`, `Texts`, `Files`, `Strings` and `Math` are recognised, but only the `Write...` / `Out...` procedures in the I/O table above are implemented. `Math.sqrt(x)` and friends fail to link.
 - **Sets are handles** into a runtime table of 0..255 bitsets, so `b := a` aliases rather than copies, and `INCL` / `EXCL` mutate every name bound to that set.
 - **No input, no files:** `Read`, `ReadInt`, `ReadString`, `Texts.Scanner`, `Files` are not implemented.
 - Not implemented: `WITH` (the keyword is recognised by the scanner but there is no statement for it), `DEFINITION` / `IMPLEMENTATION` modules and real separate compilation (a program is one `MODULE`), `IS` /type guards `v(T)`, `PROCEDURE` types and procedure variables, opaque and enumeration types, subranges, `DISPOSE` / `DEALLOCATE` (memory is never released), `SYSTEM`, coroutines, exceptions, `ASSERT`, `LONGREAL` as a distinct type, and run-time range checking.
 - The driver emits an executable only — there is no `--dll` flag, so there is no C#/VB.NET interop activity here (see `ada/ada.md` or `lua/lua.md` for languages that have one).
-

Tutorial

Every example was compiled and run with `oberon`; the output shown is real.

1. Your first program

Oberon-2 style:

```
MODULE Hello;
IMPORT Out;
BEGIN
  Out.String("Hello from Oberon-2"); Out.Ln
END Hello.
```

```
Hello from Oberon-2
```

Modula-2 style, in the same compiler:

```
MODULE Hello2;
FROM InOut IMPORT WriteString, WriteLn;
BEGIN
  WriteString("Hello from Modula-2"); WriteLn
END Hello2.
```

```
Hello from Modula-2
```

A compilation unit is one `MODULE Name; ... END Name.` — the trailing name must match and the `.` ends the file. Everything between `BEGIN` and `END` becomes `main`. The `IMPORT` clause is documentation here: the built-in output procedures are always available.

2. Variables and data types

```
MODULE Vars;
IMPORT Out;
CONST
  Max = 10;
  Pi  = 3.14159;
  Name = "Oberon";
VAR
  n: INTEGER;
  x: REAL;
  c: CHAR;
  ok: BOOLEAN;
BEGIN
  n := 42;  x := 2.5;  c := CHR(65);  ok := TRUE;
  Out.String("n    = "); Out.Int(n, 0); Out.Ln;
  Out.String("x    = "); Out.Real(x, 0); Out.Ln;
  Out.String("c    = "); Out.Char(c); Out.Ln;
  Out.String("ok   = "); Out.Int(ok, 0); Out.Ln;
  Out.String("Name = "); Out.String(Name); Out.Ln;
  Out.String("Max * 2 = "); Out.Int(Max * 2, 0); Out.Ln;
  Out.String("Pi      = "); Out.Real(Pi, 0); Out.Ln;
  Out.String("7 DIV 2 = "); Out.Int(7 DIV 2, 0); Out.Ln;
  Out.String("7 / 2   = "); Out.Real(7 / 2, 0); Out.Ln;
  Out.String("ABS(-3) = "); Out.Int(ABS(-3), 0); Out.Ln;
  Out.String("ODD(7)  = "); Out.Int(ORD(ODD(7)), 0); Out.Ln;
  Out.String("CAP(z)  = "); Out.Char(CAP(CHR(122))); Out.Ln;
  Out.String("ENTIER  = "); Out.Int(ENTIER(2.7), 0); Out.Ln;
  Out.String("FLOAT(3) = "); Out.Real(FLOAT(3), 0); Out.Ln
END Vars.
```

```
n      = 42
x      = 2.5
c      = A
ok     = TRUE
Name   = Oberon
Max * 2 = 20
Pi      = 3.14159
7 DIV 2 = 3
7 / 2   = 3.5
ABS(-3) = 3
ODD(7)  = 1
CAP(z)  = Z
ENTIER  = 2
FLOAT(3) = 3
```

Every declaration section is optional and may repeat. Note `c := CHR(65)` — there is no character literal in this dialect, because `"A"` and `'A'` both scan as strings. `DIV` truncates; `/` always produces a `REAL`. `ORD` converts a `CHAR` or `BOOLEAN` to its integer value, `FLOAT` widens an integer, and `ENTIER` / `TRUNC` narrow a real.

3. Flow control

```
MODULE Flow;
IMPORT Out;
VAR i, sum: INTEGER;
BEGIN
    sum := 0;
    FOR i := 1 TO 10 DO sum := sum + i END;
    Out.String("sum 1..10 = "); Out.Int(sum, 0); Out.Ln;

    FOR i := 1 TO 10 BY 3 DO Out.Int(i, 4) END; Out.Ln;

    i := 1;
    WHILE i < 100 DO i := i * 2 END;
    Out.String("doubling = "); Out.Int(i, 0); Out.Ln;

    i := 0;
    REPEAT INC(i) UNTIL i ≥ 3;
    Out.String("repeat = "); Out.Int(i, 0); Out.Ln;

    i := 0;
    LOOP
        INC(i);
        IF i > 4 THEN EXIT END
    END;
    Out.String("loop = "); Out.Int(i, 0); Out.Ln;

    FOR i := 1 TO 4 DO
        IF i = 1 THEN Out.String("one")
        ELIF i = 2 THEN Out.String("two")
        ELSE Out.String("many")
        END;
        Out.String(" ");
        CASE i OF
            1: Out.String("case one")
            | 2, 3: Out.String("case two or three")
            ELSE Out.String("case other")
            END;
        Out.Ln
    END
END Flow.
```

```
sum 1..10 = 55
  1   4   7  10
doubling = 128
repeat   = 3
loop     = 5
one case one
two case two or three
many case two or three
many case other
```

Every compound statement is bracketed by its keyword and `END`, so no `BEGIN` / `END` pairs are needed inside `IF` or `WHILE`. `BY` sets the step, and may be negative to count down (`FOR i := 10 TO 1 BY -1`). `LOOP` / `EXIT` is the unconditional loop with a mid-body exit. `CASE` arms are separated by `|`, and `ELSE` is the catch-all.

4. Structured types — arrays, records and sets

```
MODULE Structured;
IMPORT Out;
TYPE
  Point = RECORD x, y: INTEGER END;
  Box   = RECORD lo, hi: Point END;
VAR
  a: ARRAY 5 OF INTEGER;
  m: ARRAY 3 OF ARRAY 3 OF INTEGER;
  p: Point;
  b: Box;
  s, t, u: SET;
  i, j: INTEGER;
BEGIN
  FOR i := 0 TO 4 DO a[i] := (i + 1) * (i + 1) END;
  Out.String("squares:");
  FOR i := 0 TO 4 DO Out.Int(a[i], 4) END; Out.Ln;

  FOR i := 0 TO 2 DO
    FOR j := 0 TO 2 DO m[i][j] := (i + 1) * (j + 1) END
  END;
  FOR i := 0 TO 2 DO
    FOR j := 0 TO 2 DO Out.Int(m[i, j], 4) END;
    Out.Ln
  END;

  p.x := 3; p.y := 4;
  Out.String("p = ("); Out.Int(p.x, 0); Out.String(","); Out.Int(p.y, 0);
  Out.String(")"); Out.Ln;
  b.lo := p; b.hi.x := 10; b.hi.y := 8;
  Out.String("width = "); Out.Int(b.hi.x - b.lo.x, 0); Out.Ln;

  s := {1, 3, 5, 7};
  t := {5..9};
  u := s + t;
  Out.String("union      :");
  FOR i := 0 TO 10 DO IF i IN u THEN Out.Int(i, 3) END END; Out.Ln;
  u := s * t;
  Out.String("intersection:");
  FOR i := 0 TO 10 DO IF i IN u THEN Out.Int(i, 3) END END; Out.Ln
END Structured.
```

```
squares:  1   4   9  16  25
          1   2   3
          2   4   6
          3   6   9
p = (3,4)
width = 7
union      :  1  3  5  6  7  8  9
intersection:  5  7
```

Arrays are 0-based, unlike Pascal's declared bounds; `m[i][j]` and `m[i, j]` are the same thing. Records nest and assign whole (`b.lo := p`). Set constructors use braces and accept ranges (`{5..9}`); `+` is union, `*` intersection, `-` difference, and `IN` tests membership.

5. Subroutines and functions

```
MODULE Subs;
IMPORT Out;
VAR a, b: INTEGER;

PROCEDURE Square(k: INTEGER): INTEGER;
BEGIN
    RETURN k * k
END Square;

PROCEDURE Fact(k: INTEGER): INTEGER;
BEGIN
    IF k ≤ 1 THEN RETURN 1 ELSE RETURN k * Fact(k - 1) END
END Fact;

PROCEDURE Swap(VAR x, y: INTEGER);
VAR t: INTEGER;
BEGIN
    t := x; x := y; y := t
END Swap;

PROCEDURE Rule(w: INTEGER);
VAR i: INTEGER;
BEGIN
    FOR i := 1 TO w DO Out.Char(CHR(45)) END;
    Out.Ln
END Rule;

BEGIN
    Out.String("Square(7) = "); Out.Int(Square(7), 0); Out.Ln;
    Out.String("Fact(6)   = "); Out.Int(Fact(6), 0); Out.Ln;
    a := 1; b := 2;
    Swap(a, b);
    Out.String("after Swap: a="); Out.Int(a, 0);
    Out.String(" b="); Out.Int(b, 0); Out.Ln;
    Rule(12)
END Subs.
```

```
Square(7) = 49
Fact(6)   = 720
after Swap: a=2 b=1
```

A result type after the parameter list makes a function; `RETURN expr` yields the value. A `VAR` parameter is by reference, so `Swap` really exchanges `a` and `b`. Each `PROCEDURE` repeats its name after `END`.

6. Memory management

Records and arrays are static or stack storage; the heap is reached through `POINTER TO` and `NEW`:

```
MODULE Memory;
IMPORT Out;
TYPE
  Node    = POINTER TO NodeDesc;
  NodeDesc = RECORD value: INTEGER; next: Node END;
VAR head, n: Node; i: INTEGER;
BEGIN
  head := NIL;
  FOR i := 1 TO 3 DO
    NEW(n);           (* heap-allocate one NodeDesc *)
    n.value := i * 10; (* n.f auto-dereferences n^ *)
    n.next := head;
    head := n
  END;
  n := head;
  WHILE n # NIL DO
    Out.String("node "); Out.Int(n.value, 0); Out.Ln;
    n := n.next
  END
END Memory.
```

```
node 30
node 20
node 10
```

`NEW(p)` allocates one instance of `p`'s pointed-to type (and, if it is an extensible record, installs its vtable pointer). Note the Oberon convention of naming the pointer type and the record separately, and that `n.value` implicitly dereferences — `n^.value` also works. There is **no** `DISPOSE`: allocations live in `cc`'s flat byte arena for the life of the program, so this front end effectively leaks. Underneath, an Oberon pointer is an integer offset into that arena, which is what makes it expressible in verifiable IL.

7. Text

Text handling is deliberately thin: string constants and literals are real text, while an `ARRAY OF CHAR` is a character array you drive one element at a time.

```

MODULE Text;
IMPORT Out;
CONST Greeting = "Hello";
      Who      = "Oberon";
VAR buf: ARRAY 8 OF CHAR;
    i: INTEGER;
BEGIN
  Out.String(Greeting + ", " + Who); Out.Ln;
  Out.String("LEN(Greeting) = "); Out.Int(LEN(Greeting), 0); Out.Ln;
  IF Greeting = "Hello" THEN Out.String("compares equal"); Out.Ln END;

  buf[0] := CHR(79); buf[1] := CHR(75); buf[2] := CHR(0);
  Out.String("buf: ");
  i := 0;
  WHILE buf[i] # CHR(0) DO Out.Char(buf[i]); INC(i) END;
  Out.Ln
END Text.

```

```

Hello, Oberon
LEN(Greeting) = 5
compares equal
buf: OK

```

+ concatenates string constants, `= / < / >` compare them, `LEN` measures one. The `buf` loop is the honest shape of `ARRAY OF CHAR`: assign characters with `CHR`, terminate with `CHR(0)`, and print with `Out.Char` — `Out.String(buf)` would print a number (see *Subset boundaries*).

8. Drawing a picture — a text chart

```

MODULE Chart;
IMPORT Out;
VAR data: ARRAY 4 OF INTEGER;
    i, j: INTEGER;
BEGIN
  data[0] := 3; data[1] := 6; data[2] := 2; data[3] := 5;
  FOR i := 0 TO 3 DO
    Out.Int(i, 2); Out.String(" | ");
    FOR j := 1 TO data[i] DO Out.Char(CHR(35)) END;
    Out.String(" "); Out.Int(data[i], 0); Out.Ln
  END
END Chart.

```

```

0 | ### 3
1 | ##### 6
2 | ## 2
3 | ##### 5

```

An outer loop per row, an inner loop emitting `CHR(35) ( # )` that many times, and `Out.Int(i, 2)` to right-align the labels in a two-column field.

9. Record extension and type-bound procedures

Oberon-2's object model: extend a record, bind procedures to it, and every such procedure dispatches through the object's vtable.

```

MODULE Shapes;
IMPORT Out;
TYPE
  Figure      = POINTER TO FigureDesc;
  FigureDesc = RECORD tag: INTEGER END;
  Square      = POINTER TO SquareDesc;
  SquareDesc = RECORD (FigureDesc) side: INTEGER END;
  Circle      = POINTER TO CircleDesc;
  CircleDesc = RECORD (FigureDesc) r: INTEGER END;

PROCEDURE (f: Figure) Area(): INTEGER;
BEGIN RETURN 0 END Area;

PROCEDURE (f: Figure) Show;
BEGIN Out.String("area = "); Out.Int(f.Area(), 0); Out.Ln END Show;

PROCEDURE (s: Square) Area(): INTEGER;
BEGIN RETURN s.side * s.side END Area;

PROCEDURE (c: Circle) Area(): INTEGER;
BEGIN RETURN 3 * c.r * c.r END Area;

VAR sq: Square; ci: Circle; f: Figure;
BEGIN
  NEW(sq); sq.side := 5;
  NEW(ci); ci.r := 10;
  sq.Show;
  ci.Show;
  f := sq; f.Show;
  f := ci; f.Show
END Shapes.

```

```
area = 25
area = 300
area = 25
area = 300
```

`SquareDesc` extends `FigureDesc`, so a `Square` is assignable to a `Figure`. `Show` is bound only to `Figure` and is never overridden, yet it prints 25 then 300 — the `Area()` it calls is resolved through the vtable that `NEW` installed, so the most-derived implementation runs. That is the whole of Oberon-2's polymorphism: extension plus type-bound procedures, no `class` keyword required.

From here the natural extensions are real string variables and separate compilation of modules with `DEFINITION` / `IMPLEMENTATION` parts (see *Subset boundaries*).